McBidAsk recalc-on-load and daily tick chunking: implementation spec

Repo: ettiennehugo/MultiCharts.Net. **Base:** `origin/main` 33a51bf (merge of PR #76). All line numbers refer to that commit. **Scope:** two fixes in one new PR, kept apart from every other branch (see the coder brief). PriceAction is not touched. **Author:** Grok Bot (spec only, no code). **Status:** draft for Ettienne. Nine API points (P-RC1..P-RC9) were open. The `_McApiDump2` output answered P-RC5, P-RC6, P-RC8 and P-RC9, and the `_McRecalcProbe` run (2026-11-01, historical @ES 1-min, `realTime(False)`, `autoTrading(False)`) answered P-RC1, P-RC2, P-RC3, P-RC4 and P-RC7 (§8). All nine passed, so the defaults stand (event-driven recalc primary, `McRecalcFromCallback = false`). P-RC-EResolution is answered by Ettienne's `EResolution` dump (2026-11-01 15:56): the exact time-based set is Second/Minute/Hour/Day (§7.3).

---

1. Evidence (log 01M6C29N9KQHKWP9Z2V017GD3Q.csv, runs 180c9976 and f625f122, @ES 1-min, chart 2026-08-31 → 2026-09-18)

Observation	Log line(s)	Cause
`OF McBidAsk diag request ... chartType(Regular) from(2026-08-30 17:02) to(2026-11-02 12:19)`	97	The request `To` is `DateTime.Now+1d`, which is 6 weeks past the chart's last bar. `chartType` prints `Regular` even though `CreateFootPrint` was used (§7.1).
`OF T9 session(..) bars(13xx) mcCover(0.0)` and `OF tier McBidAsk->Unavailable reason(missing)` for every session	100–126	The McBidAsk history arrives only after the backtest has passed all bars (see the next row).
B: `RUN-END ... 16:00` comes first, then `history done batches(1) data2Bars(61837) levels(479517)`	85–92	The loader finishes after the last bar, so the levels are never used. Nothing triggers a second pass.
B: 3× `OF tick loader callback exception(System.ApplicationException) message(Array dimensions exceeded supported range.)`	86–88	Each of the Trade/Bid/Ask 1-tick requests covers 2026-08-30 → 2026-11-02 (~45 Globex days of ES ticks), which is more than one .NET array can hold. All tick history is lost.
C (f625f122): **no** callback, exception, `history done` or `stop` lines after RUN-END (the last lines are 128–130)	–	The queued callback lines drain only in `CalcBar`/`StopCalc` (`_VolumeProfileFuturesBase.Strategy.CS` 138–145, 627–639). A finished historical chart gets no new bar, so the lines sit in the queue until the study is removed. The log cannot show whether run C's callbacks have fired.
`OF MaxBars cap 5000 hit at bar 5051`	107	See §7.4.

2. MC .NET API facts used (with sources)

Fact	Source
`IStudy.ExecControl` is `ICalculationControl` and "allows the user to interrupt the calculation, recalculate the study, or force the study to calculate over a given period of time".	PLStudiesProxy metadata (`specs/mc-portfolio-api.txt` l.176); PG v1.0 §4.1.2 (l.690)
`ExecControl.Recalculate()` restarts the study from `StartCalc` and then runs `CalcBar` over all bars. Statements after it in the same method do not run, so it unwinds the stack. Calling it unconditionally loops forever, so it needs a persistent guard.	PG VolumeProfile section, txt l.3616/3700 (`this.ExecControl.Recalculate()` from the `EChanged` handler); MC wiki RecalcLastBarAfter / Recalculate; forum (recalc guard via persistent vars)
`StartCalc` "can be called repeatedly during the object's life cycle". `Create` runs once. The object instance is reused, so **plain C# fields survive a Recalculate** unless `StartCalc`/`StopCalc` reset them.	PG §4.1.2; MC wiki How Indicators and Signals are Calculated
MC uses internal exceptions for its own control flow (MaxBarsBack restart). Foreign exceptions must not be caught.	MC wiki (same page)

Fact	Source
`ExecControl.RecalcLastBarAfter(TimeSpan)` makes MC call `protected override void OnRecalcLastBarAfterEvent()` after the timeout, **even when no ticks arrive**. It does not recalculate by itself.	MC wiki; forum t=43339 (indicators). Probe P-RC2: also fires on a **signal** on a historical (non-RT) chart, about 1.0 s after `RecalcLastBarAfter(1 s)` was armed at the last bar; forum t=49867 ("does not run when backtesting") does not apply to chart backtesting. The repo already calls it from a signal: `_CustomSignal.Strategy.CS` 2068.
DataLoader callbacks run on a separate thread. `Bars`, `Output`, `trace()` and `engineSink` belong to the calc thread only.	PG §4.7.4; repo comment 712–713; probe P-RC7: a callback (tid 61) arrived while `CalcBar` (tid 60) slept in a wait loop (done in 52 ms)
The same study object (and its instance fields) is kept across `Recalculate()`.	Probe P-RC1: `obj(719710)` on both passes; `instRecalcs=1` carried into pass 2
`OnRecalcLastBarAfterEvent` runs on the calc thread, and `Bars.Time[0]` is readable in it.	Probe P-RC3: tid 60, the same as `CalcBar`
`Recalculate()` called from `OnRecalcLastBarAfterEvent` does not return: it throws `ManagedStudies.details._ELAPI_exception_Wrap_`, MC's unwind, and `StopCalc` then `StartCalc` (pass 2) follow at once. It must propagate (last statement, never caught); code after `base.CalcBar()` in a derived strategy does not run on that path.	Probe P-RC4
`StopCalc` runs on the UI thread (tid 1), **not** the calc thread. Nothing reached from `StopCalc` may read `Bars`, `Output` or other calc-thread-only API; trace lines there are stamped with the bar time cached on the calc thread.	Probe (2026-11-01), StopCalc line
`InstrumentDataRequest`: `Resolution`, `QuoteField`, `From`/`To` (the PG calls these "outdated, better to use Range"), `DataRequest Range`, `RequestTimeZone TimeZone` (Local/Exchange/GMT), `Subscribe2RT`. `Bars.Request` returns a copy of the chart's request (same feed, session and time zone).	PG §4.7.4 (l.2979–3045, 3083–3131)
`DataRequest.CreateFromTo(DateTime, DateTime)` is typed and static.	Already used in `_CheckData.Strategy.CS` 184
`Bars.FullSymbolData` indexes "from the current bar, leftwards with positive indices, and rightwards with negative ones", and `.Count` is the total bar count. The last chart bar is therefore `Time[Current - Count]`.	PG §4.1.2 (l.931–945)
Footprint levels need `Resolution.CreateFootPrint(EResolution, size, CumulativeDeltaChartType, FootPrintChartType, bool)`. Without it, `Data2` stays null.	Forum t=52737 (in code since #74); Data2 delivered 61837 bars in run B

Anything not in this table is unverified and is listed in §8. Of the §8 points, the API dump answered P-RC5, P-RC6, P-RC8 and P-RC9, and the probe answered P-RC1, P-RC2, P-RC3, P-RC4 and P-RC7.

3. Code map (origin/main 33a51bf)

`_VolumeProfileFuturesBase.Strategy.CS` - 127–136 `StartCalc`: calls `startOrderFlow()` (no trace possible here, per comment 135). - 138–145 `CalcBar`: `drainFlowQueue()` first, then `updateOrderFlow()`. - 589–609 config and fields: `FlowCfg` (protected `_OrderFlow.FlowConfig`), `McBidAskChartType` (606), `m_mc*` state. - 614–625 `startOrderFlow`: **ends all loads and resets every McBidAsk field, including `m_mcHistoryDone`. It also replaces `m_flow`, which is where the levels live.** A recalc therefore loses everything today. - 627–639 `StopCalc` and 641–648 `stopOrderFlowLoads` (`EndLoadData` on everything). - 650–673 `startFlowLoader`: the three whole-range 1-tick requests (662–664). **This is Fix 2's target.** - 677–698 `startMcLevelLoader`: the CreateFootPrint request; `To = Now+1d` at 689; the request diag line at 696. - 700–739: diag counters, `mcDiag`, the queue (`enqueueFlowMsg`, `queueSink`, `drainFlowQueue`). - 743–794 `onMcLevels`; 796–813 `mcSet` (writes only into `m_flow`); 816–841 `mcJournal`. - 844–865 `onFlowData` (the exception at 864 is the source of the three "Array dimensions" lines); 869–875 `flowTick`. - 877–921 `updateOrderFlow`: starts the loader on the first call (888). The all-or-nothing tick gate is at 910.

`_OrderFlow.Function.CS` (MC-free; the `// <order-flow>` block 30–519 is extracted by `tests/OrderFlowTests`) - 71–90 `FlowConfig` (`MaxBars = 5000` at 80). - 175–187 `Ev` and `Queue` (each stream must be pushed in time order). - 193–361 `FlowEngine`: `AddMcLevels` 226; `takeMc/applyMc` 234–270; **`CloseBar` 294–361 pops every queued trade with `Time <= CloseTime` into the current bar (301–319)**. See the backlog bug in §7.3.

`_CmeSessionClock.Function.CS`: `ToEastern` 142–154, `GlobexDate` 164, `GlobexTradingDayEt` 235, `IsGlobexClosedDay` 368, `DefaultHaltStart` 17:00 ET (58). `_CustomSignal.Strategy.CS`: logger `WriteTrace` (one `File.AppendAllText` per line under a lock) 110–125; `trace` ~360; `engineSink` 1831; RUN-END 2131–2139; `RecalcLastBarAfter` for account loading 2068. `tools/sem/stubs/McStubs.cs`: `IExecControl` has only `RecalcLastBarAfter` (57); `DataRequest` has no `CreateFromTo` (36); `InstrumentDataRequest` has no `TimeZone` (37).

4. Fix 1: cache the McBidAsk history and recalculate once on load

4.1 Pure parts (in the `<order-flow>` block, unit-tested)

- **`McLevelCache`** (sealed class):
- `SortedDictionary<DateTime, double[][]>` holding the same `{price, ask, bid}` triple as `AddMcLevels`.
- `string Signature`, `bool HistoryDone`, `DateTime First, Last`, `int Sets`, `long Levels`.
- `Add(DateTime, double[], double[], double[])`, `ReplayInto(FlowEngine)`, `Clear()`.
- `bool Covers(DateTime from, DateTime to)` (HistoryDone && First ≤ from+1 bar && Last ≥ to−1 bar).
- **`McRecalcGate`** (sealed class, no MC types). Fields: `int RecalcCount`, `string RecalcSignature`, `int MissedBeforeDone`, `DateTime MissedFirst/MissedLast`, `int Pass`.
- `NoteMissed(DateTime)`.
- `Decide(bool historyDone, bool cacheCoversMissed, bool autoTrading, bool enabled, int max, string sig)` returns a typed `enum RecalcDecision { Wait, Recalc, SkipNoMissed, SkipGuard, SkipAutoTrading, SkipDisabled }`.
- The guard: `RecalcCount < max` **for this signature**. If a pass 2 request ever re-fired its history, `SkipGuard` stops a second recalc.
- `Forget()` sets `RecalcSignature = null; RecalcCount = 0` (missed-bar counters and `Pass` are kept). It is called on a **signature** cache miss (§4.3), see "Gate reset" below.
- `SkipLine(d, sig, max)` builds the skip journal line of §4.4 (reason, `recalcCount(CountFor(sig)/max)`, `recalcSig(RecalcSignature|none)`).
- **`McLevelCache.Miss(sig, from, to)`** returns a typed repo enum `McCacheMiss { None, Empty, Signature, Incomplete, Coverage }`; `MissName` gives the journal text and `MissReason` (string) wraps the two. A cache that holds **another** request's signature is a `Signature` miss even when it holds no level set (checked before `Empty`).
- **Gate reset (A → B → A, Live Check 9a rerun).** The gate is per object and used to keep one signature for its lifetime, so A (recalc) → B (no recalc, e.g. `noMissedBars`) → A refused the recalc for the fresh A download with `SkipGuard`. Now a `Signature` miss calls `m_mcGate.Forget()` under `m_flowLock` (only when the gate holds a signature; the miss line then ends `recalcGate(reset)`). A real recalc loop comes back with the **same** signature (a hit, or an `incomplete`/`coverage` miss), so it never reaches `Forget`; the static process-wide `McRecalcLoopGuard` (`McRecalcGuardMaxPerKey` = 3 per `McRecalcGuardWindowSec` = 600 per key) stays the backstop, also for a fast A ↔ B alternation.

4.2 Field lifetime (the core of the fix)

- Add `private readonly McLevelCache m_mcCache = new McLevelCache();` and `private readonly McRecalcGate m_mcGate = new McRecalcGate();` to `_VolumeProfileFuturesBase`, next to 607–609.
- **`startOrderFlow` (614–625) must not clear either of them.**
- `StopCalc` (627–639) does not clear them either. It still ends the loads.
- They are cleared only when the signature no longer matches (§4.3), so a user-driven reload with a different symbol, resolution or chart start starts clean.
- P-RC1 (§8) confirms that the instance survives `Recalculate`.

- If the probe shows MC re-creates the object instead, the fallback is a `static Dictionary<string, McLevelCache>` keyed by signature and process-local. It holds the last 2 signatures at most.

4.3 Flow

1. **First CalcBar, at the point where `startFlowLoader` runs today (888).** Build the signature on the calc thread: `Bars.Info.Name | Bars.Info.DataFeed | res.Type | res.Size | McBidAskChartType | FlowCfg.McBidAsk | firstBarTime`. Compute `lastBarTime = Bars.FullSymbolData.Time[Current - Count]`. - **Hit** when `Signature == sig && m_mcCache.Covers(firstBarTime, lastBarTime)`:
   - Call `m_mcCache.ReplayInto(m_flow)` under `m_flowLock`.
   - Set `m_mcHistoryDone = true`, `m_mcLoaded = Sets`, `m_mcFirst/Last`.
   - **Do not call `startMcLevelLoader`**, except on a live chart (`Environment.IsRealTimeCalc` or `lastBarTime` ≥ Now−1 day), which gets a tail request instead: `Range = DataRequest.CreateFromTo(m_mcCache.Last, Now+1d)`, `Subscribe2RT = true`. It covers RT and the bars since the cache was filled and is logged as `request(tail)`.
   - Log `cache hit` and `recalc pass using cache`.
   - **Miss** (any other case): on a typed `McCacheMiss.Signature` miss with `m_mcGate.RecalcSignature != null`, `m_mcGate.Forget()` (§4.1 gate reset); then `m_mcCache.Clear(); m_mcCache.Signature = sig` (all under `m_flowLock`), log `cache miss reason(..)`, and issue the normal request (with the `To` clamp of §4.5).
2. **Callback (`onMcLevels` 743–794, `mcSet` 796–813).** `mcSet` writes each set into `m_mcCache` **and** into `m_flow` (under `m_flowLock`, which is already held). When History is completed: `m_mcCache.HistoryDone = true` (next to 790). Only state is set; **no MC call from this thread**.
3. **`mcJournal` (816–841).** When `fp.McMiss == "missing" && !m_mcHistoryDone`, call `m_mcGate.NoteMissed(fp.CloseTime)` to count the pass-1 bars that would have used McBidAsk.
4. **Trigger. This runs on the calc thread only and is factored into `private void mcRecalcCheck(string from)`.** It is called at the end of `CalcBar` (after `updateOrderFlow`) and from `OnRecalcLastBarAfterEvent`. - When `m_mcHistoryDone && m_mcGate.Decide(...) == Recalc`:
   - Log `recalc triggered`, then `drainFlowQueue()`.
   - Set `m_mcGate.RecalcCount++; m_mcGate.RecalcSignature = sig`.
   - **Then `ExecControl.Recalculate();` as the last statement, not inside any `try/catch`** (it unwinds; see §2).
   - `cacheCoversMissed` = `m_mcCache.First ≤ MissedLast && m_mcCache.Last ≥ MissedFirst`.
   - `SkipNoMissed` means the history arrived before any bar needed it, so no recalc is needed.
   - `SkipAutoTrading`: never auto-recalc while `Environment.IsAutoTradingMode`, because orders would be re-evaluated. Log the skip once.
5. **Idle poll.** This is what makes the trigger, and the log drain, work with no new bar. - In `CalcBar`, when `Bars.LastBarOnChart && Bars.Status == EBarState.Close` and either (a) `m_mcStarted && !m_mcHistoryDone`, or (b) the flow queue is non-empty, or (c) a tick chunk is pending (Fix 2):
   - call `ExecControl.RecalcLastBarAfter(TimeSpan.FromMilliseconds(FlowCfg.McIdlePollMs))` once;
   - set `m_idleArmed = true`;
   - log `idle poll armed` once.
   - New override in `_VolumeProfileFuturesBase`: `protected override void OnRecalcLastBarAfterEvent() { base.OnRecalcLastBarAfterEvent(); // keep whatever the base does (_CustomSignal arms it for account loading, 2068) drainFlowQueue(); pumpTickChunks(); // Fix 2 if (mcRecalcDecision() == RecalcDecision.Recalc) { mcRecalcNow("idle"); return; } // mcRecalcNow ends with ExecControl.Recalculate() if (!idleWorkPending()) return; if ((DateTime.Now - m_idleSince).TotalSeconds < FlowCfg.McIdleTimeoutSec) ExecControl.RecalcLastBarAfter(TimeSpan.FromMilliseconds(FlowCfg.McIdlePollMs)); else logOnce("OF McBidAsk idle poll timeout ..."); }`
   - `drainFlowQueue` stamps lines through `trace()` with `Bars.Time[0]`. This is legal only if the event runs in calc context (P-RC3).
6. **Fallback if P-RC2/P-RC3 fail** (they passed, so this stays off by default) (the event does not fire on a historical signal, or `Bars` is not accessible in it). Add `FlowCfg.McRecalcFromCallback = false`. When it is `true`, the history-completed branch of `onMcLevels` calls `ExecControl.Recalculate()` directly once, guarded by the gate. The precedent is the PG VolumeProfile `EChanged` handler (txt

l.3616) and the repo's `_VolumeProfileStatistics.Indicator.CS` 357. - The queued lines then get a direct-write path: snapshot `Bars.Time[0]`, the zone label and `RunId` in each `CalcBar` into volatile fields, and add a protected `_CustomSignal.traceAt(ETraceLevel, DateTime stamp, string msg)` (the logger is private, at 62) that callbacks call with the snapshot. That is still one `File.AppendAllText` per line. Mark these lines with `async(1)`. - Ship this path disabled. The probe decides the default.

4.4 Diag lines (Journal; prefix `OF McBidAsk`, so the `_VolumeProfileBaseLog` parsers keep working)

```
OF McBidAsk cache miss reason(empty|signature|incomplete|coverage) sig(<sig>) [recalcGate(reset)]
OF McBidAsk cache stored sets(N) levels(L) first(yyyy-MM-dd HH:mm) last(..) sig(<sig>)          (calc thread
OF McBidAsk cache hit sets(N) levels(L) first(..) last(..) pass(P) request(none|tail from(..))
OF McBidAsk recalc triggered from(bar|idle|callback) pass(P) missedBars(M) missedFirst(..) missedLast(..) re
OF McBidAsk recalc pass using cache pass(P) recalcCount(1)
OF McBidAsk recalc skipped reason(noMissedBars|guard|autoTrading|disabled) recalcCount(n/McRecalcMax) recalc
OF McBidAsk recalc skipped reason(processGuard) maxPerKey(3) windowSec(600) key(<key>)
OF McBidAsk idle poll armed ms(1000)   /   OF McBidAsk idle poll timeout sec(900) historyDone(False) chunksP
```

`recalcCount`/`recalcSig` are read under `m_flowLock` with the decision: `reason(guard) recalcCount(1/1) recalcSig(<sig>)` is a previous recalc of this signature, `recalcCount(0/0) recalcSig(none)` is `McRecalcMax = 0`. `Pass` counts the `StartCalc` calls in the object's lifetime (a field that is never reset). The existing `RUN id(..)` line gets a new id per pass (CS-3, 2014), which is expected.

4.5 Request window clamp (applies to Fix 1 and Fix 2)

- Replace `To = DateTime.Now.AddDays(1)` (663, 689) with `lastBarTime.AddDays(1)`, where `lastBarTime` is as in §4.3.
- Set the range through `req.Range = DataRequest.CreateFromTo(from, to)` (typed, as in `_CheckData` 184), and also set `From`/`To` to the same values. P-RC5 (dump) confirmed these members: `Range` is a `DataRequest` field and `From`/`To` are properties. Setting both is harmless, so do it whichever one MC reads.
- On run B this cuts the McBidAsk load from 61837 to about 20k bars (14 Globex days × ~1380).

5. Fix 2: daily, session-aware tick chunks

5.1 Plan (pure, unit-tested; `<order-flow>` block)

- `static List<DayChunk> PlanGlobexDays(DateTime firstGx, DateTime lastGx, Func<DateTime, bool> closedDay, TimeSpan boundaryEt)` returns one `DayChunk { DateTime GlobexDate, FromEt, ToEt }` per Globex date that is not closed.
- Each window is `[GlobexDate−1 day + boundaryEt, GlobexDate + boundaryEt)`, with `boundaryEt = 17:00` (CME daily halt start, `_CmeSessionClock.DefaultHaltStart`), so no tick straddles two chunks. Monday's chunk starts Sunday 17:00 ET.
- `firstGx`/`lastGx` come from `clock().GlobexDate(firstBarTime / lastBarTime)`, and `closedDay` from `clock().IsGlobexClosedDay`.
- **Time zone.** The windows are built in ET and converted to the request zone with a new `public DateTime FromEastern(DateTime et)` on `_CmeSessionClock` (the inverse of `ToEastern` at 142; DST-aware; ambiguous → standard). The request keeps `Bars.Request`'s `TimeZone` (Exchange on 9a: CME, Central), so tick times and bar times share one zone. P-RC6 (dump): `InstrumentDataRequest` has a `RequestTimeZone TimeZone` field (`Local`, `GMT`, `Exchange`), so the copy from `Bars.Request` carries it. Log it typed in the request line.
- **`TickChunkScheduler`** (pure). Its state per (day, stream): `Pending → InFlight → Done | Failed | Skipped`.
- `Next(DateTime calcPosition)` yields up to `MaxInFlightDays` days (all streams of a day together).
- `Complete(day, stream, ok, count)` records the outcome.
- `ReleasableInOrder()` frees completed days strictly in day order per stream, because `FlowEngine.Queue` needs each stream in time order (177, 190). With `MaxInFlightDays = 1` (the default) the order is guaranteed and the release buffer is a no-op.

5.2 Requests (replace 650–667)

- For each chunk and stream (Trade, plus Bid/Ask when `QuotesExpected`):
- `req = Bars.Request; req.Resolution = new Resolution(EResolution.Tick, 1); req.QuoteField = field; req.Range = DataRequest.CreateFromTo(from, to)` (plus the same `From`/`To`).
- `req.Subscribe2RT = isLastChunk`.
- `state = chunkIndex * 3 + stream`, boxed `int` as today (848).
- **Issued only from the calc thread** by `pumpTickChunks()`, which is called in `updateOrderFlow` (replacing 888) and from `OnRecalcLastBarAfterEvent`. BeginLoadData is never called from a callback; that is unverified, so it is not used.
- **Late days.** Under `FlowCfg.TickChunkLate == Skip` (the default), a day whose window ends before the last closed bar at issue time is marked `Skipped reason(late)` and not requested, because its ticks could no longer be used. The last chunk, which carries RT, is always requested.
- **Bounded wait mode** (`FlowCfg.TickChunkWaitMs > 0`; default 0 = off). This mode makes the tick tier usable in historical backtests. At the first bar of each Globex day, `CalcBar` waits up to `TickChunkWaitMs` for that day's chunks with `Thread.Sleep(50)` polling of a volatile flag. Late days are then not skipped. P-RC7 confirms that callbacks arrive while the calc thread sleeps. Log `OF tick chunk wait day(..) ms(..) ready(True|False)`. In wait mode the strategy also counts each closed bar's `FlowSource` per session (`SessionSourceTally`, lazy, per pass, under `m_flowLock`) and logs one Info line per session when the session changes, the last one in `StopCalc`: `OF tick source session(yyyy-MM-dd) bars(n) quote(q) tickRule(t) mcBidAsk(m) aggressor(a) barUpDown(b) unavailable(u)`. Each session is reported once per pass (deduped), so a session that stays all-Quote (`tickRule(0)`) leaves a trace. No new `[Input]`.

5.3 Callback (`onFlowData` 844–865)

- Look up the chunk from `State`.
- Copy `result.Data` once into a local array inside its own `try`. Iterate under `m_flowLock` as today, but only after the chunk is releasable.
- On `IsCompleted`, set the chunk to Done and record the count.
- **Errors are per chunk.** The `catch` marks only that (day, stream) Failed and queues `OF tick chunk failed ...`. If `TickChunkSplitOnFail > 0` and this was the first attempt, the scheduler re-plans that day as N equal sub-windows (default 4 × 6 h), issued by the pump. Otherwise the day stays Failed and the next day continues.
- **The tick gate (910) becomes per bar.** A bar uses `TickStream` iff the chunks covering its close time are Done for Trade (and Bid/Ask when quotes are on). Otherwise it uses `BarsOnly`, so a failed day falls back to the bar tiers. Replace `m_flowHistoryDone[0]` with `m_chunks.ReadyAt(CloseTime)`. `onFlowData` keeps setting `m_flowHistoryDone[stream]` for the last chunk only, which keeps the RT semantics.
- **Memory guard.** When `TickChunkLate == Skip`, the callback drops the ticks of a chunk whose window ends at or before the last closed bar (volatile snapshot) instead of queuing them. They are counted as `late`.

5.4 Diag lines (Info per chunk; Journal for the plan, failures and the summary)

```
Journal  OF tick chunk plan days(D) streams(Trade,Bid,Ask) first(yyyy-MM-dd) last(..) zone(Exchange) boundar
Info     OF tick chunk start day(yyyy-MM-dd) stream(Bid) from(yyyy-MM-dd HH:mm) to(..) rt(False) attempt(1)
Info     OF tick chunk done day(..) stream(..) ticks(n) ms(t)
Info     OF tick chunk skipped day(..) reason(late|closedDay)
Info     OF tick source session(yyyy-MM-dd) bars(n) quote(q) tickRule(t) mcBidAsk(m) aggressor(a) barUpDown
Journal  OF tick chunk failed day(..) stream(..) attempt(a) exception(<FullName>) message(..) action(split4
Journal  OF tick chunk summary done(x) failed(y) skipped(z) ticks(n) lateDropped(m)      (once, when nothing
```

5.5 Config (all on the protected `FlowCfg`; no `[Input]`; set by the derived strategy as today)

Field	Default	Meaning
McRecalcOnLoad	true	Fix 1 trigger on/off

Field	Default	Meaning
McRecalcMax	1	recalcs per cache signature (loop guard)
McIdlePollMs	1000	RecalcLastBarAfter interval while idle work is pending
McIdleTimeoutSec	900	stop polling after this
McRecalcFromCallback	false	fallback §4.3 step 6 (the probe decides)
TickChunkByDay	true	false = legacy single request (for A/B only)
TickChunkMaxInFlightDays	1	1..4
TickChunkSplitOnFail	4	sub-windows for a failed day; 0 = no retry
TickChunkLate	Skip	typed enum ETickChunkLate { Skip, Load }
TickChunkWaitMs	0	bounded per-day wait in CalcBar; 0 = off
TickChunkBoundaryEt	17:00	chunk boundary (ET)

6. Threading rules (all from §2)

1. MC calls (ExecControl.*, DataLoader.BeginLoadData/EndLoadData, Bars, trace, engineSink) run only in CalcBar, StartCalc (no trace there), StopCalc and OnRecalcLastBarAfterEvent (P-RC3: calc thread). The only exceptions are McRecalcFromCallback and the existing EndLoadData paths. - StopCalc runs on the UI thread (probe). It ends loads (EndLoadData) and drains the queue, but reads no Bars/Output: _CustomSignal.trace() checks the thread and, off the calc thread, stamps the bar time, symbol and zone cached by noteCalcThread() (called first in CalcBar and OnRecalcLastBarAfterEvent) and writes no Output.
2. Callbacks touch only m_flow, m_mcCache and scheduler state under m_flowLock, plus the queue under m_flowQueueLock.
3. Recalculate() is the last statement on its path and is never wrapped in a catch-all.
4. Don't catch exceptions around base.* calls (MC's MaxBarsBack restart).

7. Code issues found (flag to Ettienne; fix in this PR unless noted)

7.1 chartType(Regular) on the "FootPrint" request (696)

- When McBidAskChartType == FootPrint, line 684 builds Resolution.CreateFootPrint(...) and logs res.ChartType, which reads **Regular**.
- The request does work: run B returned data2Bars(61837) levels(479517). So CreateFootPrint carries the footprint mode in other Resolution members (the stubs hint at a delta-type member), and ChartType stays Regular.
- The line is misleading, and so is chartType(FootPrint) in the loaded line (822), because that one prints the config value rather than the request.
- **Fix:** log builder(CreateFootPrint delta(ASK_TRADE_BID_TRADE) footprint(BID_ASK_VOL) pct(False)) resChartType(Regular) from the typed enum values passed in. Once the dump (§8) names the footprint members, log them typed as well.
- No behaviour change. P-RC8 (dump) answered this: after CreateFootPrint, ChartType = Regular, DeltaType = None and HaveDelta = True, and Size is UInt32. Regular is therefore expected. The footprint mode shows in HaveDelta, so log haveDelta(..) typed next to resChartType(..).

7.2 To = Now+1d (663, 689)

This loads 6 weeks of data past the chart end, and it is a direct contributor to the array overflow. Fixed by §4.5.

7.3 Backlog bug in FlowEngine.CloseBar (301–319). This must be fixed in commit 2, because chunking makes late arrival normal.

- CloseBar pops **every** queued trade with Time <= CloseTime. When history arrives after bars have already closed as BarsOnly, the first TickStream bar absorbs all the earlier ticks into one giant

footprint. With `McBidAsk = Prefer`, that bar's MC set then fails the volume check (`volMismatch`, 251), and the garbage footprint stays tagged Quote/High.
- The first bar also absorbs the extra day of ticks from `from = first − 1 day` (654).
- **Fix (MC-free):** the engine remembers `m_lastClose`, the previous `CloseBar`'s CloseTime. Trades with `Time <= m_lastClose` are popped and **not added**; count them in a new `fp.BacklogDropped` / `BacklogTrades` counter. The first bar uses `CloseTime − bar duration`, passed as a new `BarInput.OpenTime`.
- **Which charts get the `OpenTime` cut (P-RC-EResolution, answered).** Only **exact time-based** Regular charts: a typed `switch` on `EResolution` maps `Second`, `Minute`, `Hour` and `Day` to a unit of 1 s / 1 m / 1 h / 1 d, and the bar length is `Size × unit` (`_OrderFlow.ExactBarLength`). `Week`, `Month`, `Quarter` and `Year` are calendar-aligned, so they are **not** cut by length; they stay on the non-time path (previous-close cut only), logged once at Info (`OF tick backlog calendar resolution type() size() -> no OpenTime cut`). Every other member (Tick, Volume, Points, Change, OrigPoints, PointAndFigure, Kagi, LineBreak, Renko, HeikinAshi, FootPrint, CumDelta, Unknown, all Custom*) and every non-Regular chart type stays non-time, as before. `Duration()` is not used for the decision. Source: Ettienne's `EResolution` dump (his MC install, 2026-11-01 15:56): Unknown=0, Tick=1, Minute=2, Hour=3, Day=4, Week=5, Month=6, Year=7, Volume=8, Second=9, Quarter=10, Points=11, Change=12, OrigPoints=13, PointAndFigure=14, Kagi=15, LineBreak=16, Renko=17, HeikinAshi=18, FootPrint=19, CumDelta=20, Custom*=1001..1013; `new Resolution(e, 1).Duration()` is non-zero only for Second 1 s, Minute 1 m, Hour 1 h, Day 1 d, Week 7 d, Month 30 d, Quarter 91 d, Year 365 d (0 for all others, Custom* included); the chart check Regular/Minute/Size 1 gave 00:01:00. (This replaces the interim Minute/Day-only rule.)
- The quote queues already only advance state (287–291), so nothing changes there.

7.4 MaxBars 5000 (80, 353–359)

- The 19072-bar run drops the oldest footprints from bar 5051.
- CumDelta is **not** actually truncated with `ResetMode.Session` on 1-min ES (a reset period is ≤ ~1380 bars, which is less than 5000). Only ladder/footprint lookups more than 5000 bars back return null, and the warning text overstates this.
- **Out of scope.** Proposal for later: `MaxBars = 0` means auto = `max(5000, 2 × bars per reset period)`. Do not scale with the loaded range, because footprint memory is roughly 2–5 KB per bar.
- Trivial wording fix allowed in this PR: "…; CumDelta is Truncated only if a reset period exceeds the cap".

7.5 Smaller items

- **Separate PR, not this one.** `_CustomSignal.Strategy.CS` 1292–1310 (`M-TICK enqueueTick/tickBarMember`) reads `Bar` members **by reflection with member-name strings** ("Time", "Close", "TotalVolume"). That breaks the hard rule. Use the typed `b.Time`, `b.Close`, `(double)b.TotalVolume`, as `flowTick` 869–875 already does.
- **FYI.** `_VolumeProfileStatistics.Indicator.CS` 357 wraps `Recalculate()` in `catch { }`, which may swallow MC's unwind (§2).
- `startOrderFlow` (618) resets `m_mcResult`/counters while a callback of the previous pass may still be running. This is harmless once the cache is the source of truth, but keep the callback's `m_flow == null` check (763).
- The stubs (`McStubs.cs` 36–37, 57) need `ICalculationControl.Recalculate()`, `DataRequest.CreateFromTo`, `InstrumentDataRequest.TimeZone` and `IFullSymbolData.Count` for `tools/sem` compile checks.

8. API points: the two one-off files (P-RC1..P-RC9 answered)

The files are saved as `/workspace/specs/_McRecalcProbe.Strategy.CS` (signal) and `/workspace/specs/_McApiDump2.Indicator.CS` (indicator). Import both into the PowerLanguage .NET Editor and compile. - Put `_McApiDump2` on any @ES chart and copy the Output window. - Put `_McRecalcProbe` alone on a historical @ES 1-min chart (the 9a setup), wait about 30 s, remove it, and send `%TEMP%\McRecalcProbe.csv`.

ID	Question	Answered by
P-RC1	Same object instance (`obj`) and instance fields kept across `Recalculate`?	**Answered (probe):** yes. `obj(719710)` on both passes; `instRecalcs=1` carried into pass 2. The instance cache and gate are valid; the static guard stays as a backstop.
P-RC2	Does `OnRecalcLastBarAfterEvent` fire on a **signal** on a non-real-time chart after the last bar?	**Answered (probe):** yes, about 1.0 s after `RecalcLastBarAfter(1 s)` was armed at the last bar (historical chart, `realTime(False)`, `autoTrading(False)`). Forum t=49867 does not apply to chart backtesting.
P-RC3	Its thread vs CalcBar (`tid`), and are `Bars` readable in it?	**Answered (probe):** the calc thread (tid 60, same as `CalcBar`); `Bars.Time[0]` is readable.
P-RC4	May `Recalculate()` be called from that event, and how does it unwind (exception type)?	**Answered (probe):** it throws `ManagedStudies.details._ELAPI_exception_Wrap_` (MC's unwind); `StopCalc` and then `StartCalc` of pass 2 follow at once. It must propagate: last statement, never caught. Derived code after `base.CalcBar()` does not run on that path; `RecalcPending` stays for it.
P-RC5	`DataRequest` members (`CreateFromTo`, `To`, `ToAlwaysIsNow`, `Count`); which wins, Range or `From`/`To`?	**Answered (dump):** `DataRequest` is a struct with `CreateFromTo`/`CreateDaysBack`/`CreateBarsBack` and the fields `RequestType`, `Count`, `From`, `To`, `ToAlwaysIsNow`. `InstrumentDataRequest` has `From`/`To` properties and a field `DataRequest Range`. Set both (§5.2). The request line's logged window shows which one MC used.
P-RC6	`InstrumentDataRequest.TimeZone` exists and is copied by `Bars.Request`; `FullSymbolData.Time[Current−Count]` = last bar	**Answered (dump):** `InstrumentDataRequest` has a field `RequestTimeZone TimeZone` (`Local`, `GMT`, `Exchange`). It is a struct field, so `Bars.Request` copies it. The run's logged first/last bar line checks the last-bar index.
P-RC7	Do callbacks arrive while CalcBar sleeps (bounded wait)?	**Answered (probe):** yes. A DataLoader callback (tid 61) arrived while `CalcBar` slept in the wait loop (done in 52 ms). `TickChunkWaitMs` stays.
P-RC8	`Resolution` members after `CreateFootPrint` (why `ChartType` = Regular) and the `Size` type	**Answered (dump):** after `CreateFootPrint`, `ChartType = Regular`, `DeltaType = None`, `HaveDelta = True`; `Size` is `UInt32`. `Regular` is expected (§7.1).
P-RC9	`ICalculationControl` members and the virtual hooks on the SignalObject base chain	**Answered (dump):** `ICalculationControl` has `Abort()`, `Recalculate()` and `RecalcLastBarAfter(TimeSpan)`. `CStudyAbstract` has `virtual OnRecalcLastBarAfterEvent()`.

Decision table: if P-RC2 or P-RC3 fails, set `McRecalcFromCallback = true` as the default and enable the direct-write log path. If P-RC7 fails, remove `TickChunkWaitMs`. If P-RC5 shows that `From`/`To` win, keep setting both (harmless). **Outcome:** P-RC2, P-RC3 and P-RC7 passed, so `McRecalcFromCallback` stays `false` and `TickChunkWaitMs` stays. The probe also showed that `StopCalc` runs on the UI thread (tid 1); see §6 rule 1.

P-RC-EResolution: answered by Ettienne's `EResolution` dump (2026-11-01 15:56). Exact time-based = Regular `Second`/`Minute`/`Hour`/`Day` (bar length `Size × unit`); `Week`/`Month`/`Quarter`/`Year` are calendar-aligned and stay on the non-time fallback (logged once at Info); all other members are non-time. Details in §7.3.

Probe source (the dump is in the file):

```
// (abridged; full file /workspace/specs/_McRecalcProbe.Strategy.CS)
protected override void StartCalc() { m_pass++; m_bars = 0; m_armed = false; log("StartCalc"); }
protected override void CalcBar() {
  m_bars++;
  if (m_bars == 1) { /* log Current, Count, Time[Current-Count], Bars.Request.TimeZone; pass 1: BeginLoadDat
                       Range = DataRequest.CreateFromTo(last-2d, last), then Sleep(50)-wait up to 20 s for
  if (Bars.LastBarOnChart && Bars.Status == EBarState.Close && !m_armed) { m_armed = true; ExecControl.Recal
}
protected override void OnRecalcLastBarAfterEvent() {
  log("OnRecalcLastBarAfterEvent");                    // every log line carries tid, obj, pass, instRecalc
  try { log("Bars.Time[0](" + Bars.Time[0] + ")"); } catch (Exception ex) { log("Bars exception " + ex.GetTy
  if (m_pass == 1 && s_recalcs == 0 && m_recalcs == 0) { s_recalcs++; m_recalcs++;
    try { ExecControl.Recalculate(); } catch (Exception ex) { log("Recalculate threw " + ex.GetType().FullNa
```

```
    log("returned from Recalculate"); }
}
```

9. Tests

Off-MC unit tests (`tests/OrderFlowTests/Program.cs`, C# 7.3, run with `run.sh`): - `McLevelCache`: Add/Covers/ReplayInto produce the same `CloseBar` result as direct `AddMcLevels`. - `Clear` on a signature change. - `McRecalcGate`: - `Wait` before historyDone. - `SkipNoMissed` when no misses; `Recalc` once; `SkipGuard` on the second call with the same signature. - `SkipAutoTrading`; `SkipDisabled`. - Gate reset (AB1–AB12): A → B → A allows a recalc for the fresh A (also when B completed with no level set); a same-signature repeat is still `SkipGuard` at `McRecalcMax`; an A ↔ B alternation still trips the process guard at 3 per key per 600 s; the skip-line format; `Forget` keeps `Pass`; strategy wiring (source checks). - `SessionSourceTally` (SS1–SS7): one line per finished session, the open session on `Flush`, deduped, every source counted; strategy wiring (wait mode only, Info, StopCalc flush). - `PlanGlobexDays`: Sep 1–18 2026 gives 14 Globex days (Sep 7 Labor Day trades with an early halt; the log has `session(2026-09-07) bars(1139)`). Monday starts Sunday 17:00 ET. Windows are contiguous and non-overlapping. - `TickChunkScheduler`: - in-order release with MaxInFlight 1 and 3 (out-of-order completion); - Failed → split into 4 → Done; - Skip-late; last chunk RT; - `ReadyAt`. - Backlog fix: ticks pushed after 3 BarsOnly bars are not added to bar 4; the counter equals their count; bar 4 contains only its own ticks.

`tests/SessionClockTests`: `FromEastern(ToEastern(t)) == t` across both DST changes for Exchange/Central and Local zones.

Live acceptance (Ettienne runs, Analyst checks the log): - **A1, Fix 1 (9a rerun, @ES 1-min, Aug 31 → Sep 18, McBidAsk = Prefer)**: - exactly one `OF McBidAsk diag request` line in pass 1; - one `cache stored`; - exactly one `recalc triggered`; - a second `RUN id(..)` followed by `cache hit ... request(none)` and `recalc pass using cache`; - **no** further `diag request` line and no second `recalc triggered`; - `OF T9 session(..) mcCover(> 0)` for every session inside the loaded window, expected ≈ 100% except `volMismatch` bars; - the `diag stop` / chunk summary lines appear **without** removing the study (idle drain). - **A2, Fix 2 (same chart, Sep 1–18)**: - no `Array dimensions exceeded` anywhere; - one `tick chunk plan`; - per-chunk `start`/`done` (or `skipped reason(late)`) lines; - a `summary` with `failed(0)`. - **A3, Fix 2 wait mode (`TickChunkWaitMs = 120000`, McBidAsk = Off)**: - every day `done`; - Quote/TickRule bars present in sessions (the quality line shows source Quote; one `tick source session(..)` line per session); - no bar with a backlog footprint (`BacklogDropped` journaled only if > 0). - **A4, regression**: with `McBidAsk = Off`, `TickChunkByDay = false`, the run behaves like 33a51bf, except for the clamp and the backlog fix. - **A5, loop guard**: in a `McRecalcMax = 1` run, press Recalculate manually after A1. The result must be `cache hit`, no request, and no recalc. - **A6, A → B → A** (Live Check 9a rerun): load A (recalc), switch the chart to B, then back to A. The A line must be `cache miss reason(signature) .. recalcGate(reset)` followed by `recalc triggered`, not `recalc skipped reason(guard)`.